PRACTICAL COURSE · EDITION 1.0 · NOVEMBER 2026

Running an AI Homelab on a Budget

From a personal laptop or an old desktop to a local studio for scripts, voice-overs, images, video and editing, built for YouTube production

---

Written in plain English for people with basic computer skills. No programming background is needed to finish Chapters 1 to 13. Model names, prices and software versions change quickly. Treat every named product as an example to check on the day you buy or install.

Contents

Chapter 1 · Introduction: why a local AI studio, and how a video is made

Chapter 2 · Hardware: what really matters for AI work

Chapter 3 · Choosing a budget machine

Chapter 4 · Operating systems: Windows, Linux or macOS

Chapter 5 · Preparing the system, step by step

Chapter 6 · Installing the core AI stack and your first chat

Chapter 7 · Selecting LLM models

Chapter 8 · LLMs for deep thinking, coding and scripts

Chapter 9 · Image generation

Chapter 10 · Narration and voice (text-to-speech, speech-to-text)

Chapter 11 · Video generation: what is realistic on a budget

Chapter 12 · Video editing and final export

Chapter 13 · Automating the YouTube production line

Chapter 14 · Hardware limits, troubleshooting and performance

Chapter 15 · Career path, budget plan and 12-week study plan

Appendix A · Command cheat sheet

Appendix B · Glossary

Appendix C · Licence quick reference

CHAPTER 1

Introduction: why a local AI studio, and how a video is made

1.1 What this course is for

This course shows you how to turn an ordinary laptop or an old desktop into a private AI workstation. The machine will write scripts, read and write text with a language model, produce voice-overs, draw thumbnails and illustrations, generate short video clips, and help you edit the finished video. You will do all of this without paying for each request, and without sending your unpublished work to a cloud service.

The course is written for a common person with basic computer skills: you can install programs, copy files, and open a terminal when told. You do not need a programming degree. Where a command appears, it is explained, and you can copy it exactly.

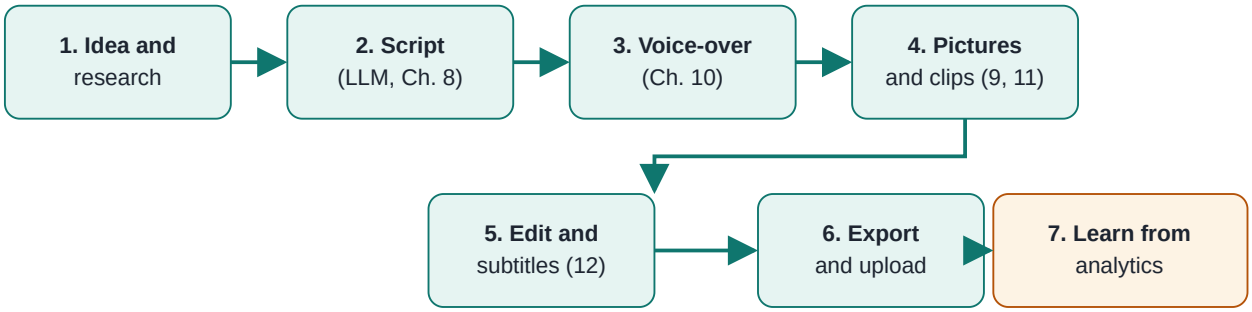
1.2 Why run AI locally?

- **Cost control.** A cloud subscription charges per month or per request. A local model costs electricity, and once the machine is paid for, every extra script or image is free.
- **Privacy.** Drafts, client material and your channel's unpublished plans stay on your disk.
- **No waiting for quotas.** You can run a thousand experiments on a Sunday night without hitting a daily limit.
- **Learning.** Running the tools yourself teaches you how models behave, which is a skill employers and clients pay for.

Local AI also has real drawbacks, which we discuss honestly in Chapter 14. The most powerful models will not fit on a budget machine, and video generation is slow. A good studio mixes local tools with occasional cloud use. The goal of this course is to make the local part as strong as your budget allows.

1.3 The YouTube production line in one picture

Almost every faceless or semi-faceless YouTube video follows the same chain of steps. Each step in this course maps to one part of the chain, so keep this picture in mind.



Every box can run on your homelab. Human checks (fact, tone, rights) sit between the boxes.

Figure 1.1 The seven stages of a YouTube video. The course covers each one.

1.4 What your homelab will and will not do well

Task	Realistic on a budget machine?	Comment
Scripts, outlines, titles, descriptions	Yes, very well	Small and medium language models are good enough when you give clear instructions.
Fact-checking your own draft	Partly	Local models can miss facts. You must verify important claims yourself.
Voice-over narration	Yes	Modern open text-to-speech runs even on a CPU. Quality is good, not studio-perfect.
Still images and thumbnails	Yes, with an 8 GB or larger GPU	Slow on weak hardware. Text inside images is still unreliable.
Short AI video clips (3 to 6 seconds)	Only with 12 GB+ GPU, and slowly	Expect minutes to tens of minutes per clip. Use as B-roll, not as the whole video.
Editing, subtitles, export	Yes	Editing needs little AI. The bottleneck is usually your own time.
Long feature-length video	No	Not practical on any budget hardware.

1.5 How this course is organised

Chapters 2 to 5 prepare you: you learn what hardware matters, choose a machine, pick an operating system, and set it up. Chapter 6 installs the software. Chapters 7 to 12 cover each capability, one at a time. Chapter 13 joins them into a repeatable production line. Chapter 14 deals with limits and problems. Chapter 15 gives a career path and a 12-week plan. The appendices hold a command sheet, a glossary and a licence guide.

> **TIP**
>
> Read Chapters 1 to 6 in order, even if you already have a computer. Most failed home AI setups go wrong in the preparation stage, not in the AI itself.

CHAPTER 2

Hardware: what really matters for AI work

2.1 The five parts that decide what you can do

1. **Graphics memory (VRAM)** on the graphics card. This is the single most important number for images, video and fast language models.
2. **System memory (RAM)**. It holds the operating system, your editing software, and any model that does not fit in VRAM.
3. **The graphics processor (GPU)**. Its speed matters, but only after VRAM is large enough.
4. **The central processor (CPU)**. It matters for video editing, decoding, and for models that run without a GPU.
5. **Storage**. Models range from a few gigabytes to over 100 GB. A fast SSD also speeds up loading.

2.2 Where the model lives: the memory ladder

An AI model is a large file of numbers. To answer a request quickly, the computer must read those numbers many times per second. The closer the numbers sit to the processor, and the faster the link, the faster the answer.

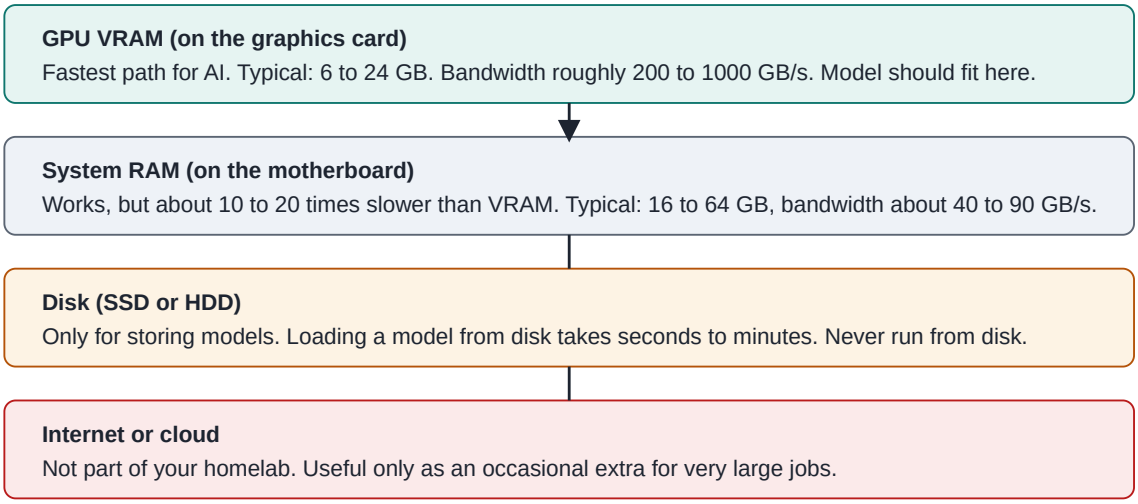


Figure 2.1 The memory ladder. Speed falls as you go down. Keep the working model in VRAM.

2.3 Why VRAM is the key number

A language model that is fully inside the graphics card answers at a comfortable speed. If part of it spills into system RAM, the answer slows down sharply. If it does not fit at all, the program either refuses to start or crashes with an "out of memory" message. Image and video models behave the same way: they need their weights plus working space for each step.

A quick rule to estimate memory

For a language model compressed to about 4 bits per weight (the most common format, called Q4), the file size is roughly:

```
file size (GB) ≈ number of parameters (billions) × 0.6

  7B model   → about 4.5 GB
  8B model   → about 5 GB
  14B model  → about 9 GB
  32B model  → about 19 to 20 GB
  70B model  → about 42 GB
```

On top of the weights, the model needs memory for the conversation so far (the *context*). A long context can add one to several gigabytes. A sensible target is to leave 1 to 2 GB free on the graphics card for this.

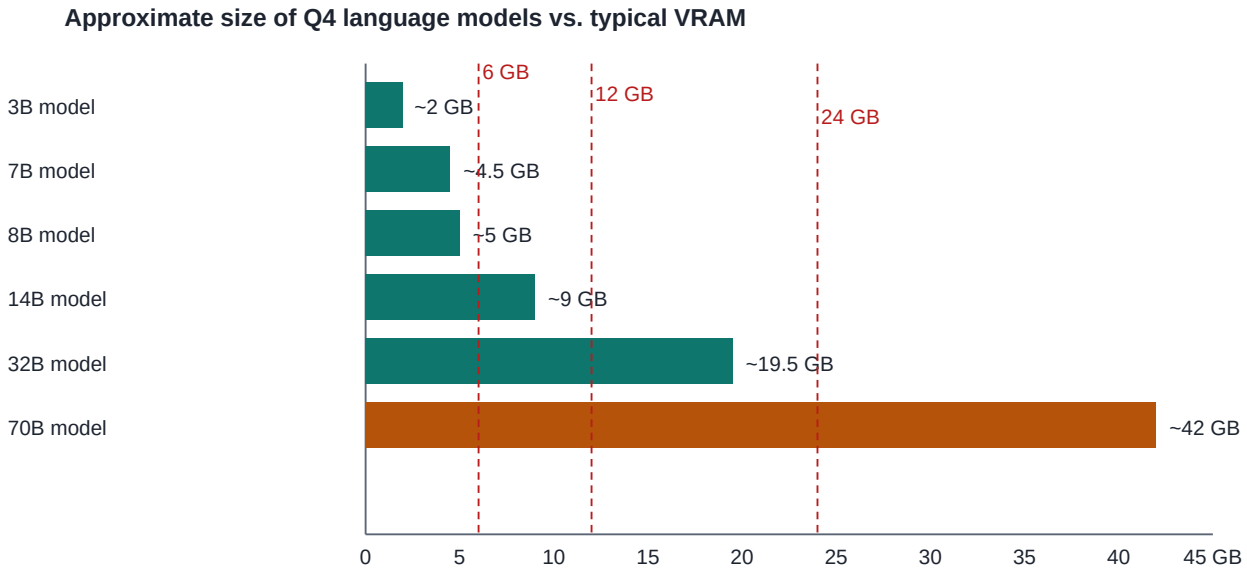


Figure 2.2 Model sizes (green = fits on a 12 GB or 24 GB card, orange = needs a 48 GB card or heavy offloading). Dashed lines show common VRAM sizes. Sizes are approximate.

2.4 Speed: why memory bandwidth matters more than the chip name

When a language model writes text, it reads almost all of its weights for each new word (token). So the speed is roughly limited by how fast memory can be read. A simple estimate:

```
tokens per second  ≈  memory bandwidth (GB/s)  ÷  model file size (GB)

Example: 7B model at 4.5 GB on a graphics card with 360 GB/s
         ≈ 80 tokens/s theoretical (real results are lower, often 40 to 60)

Example: same model on dual-channel DDR4 RAM at 50 GB/s
         ≈ 11 tokens/s theoretical (real results often 4 to 8)
```

Reading speed for a human is about 5 to 8 words per second. Anything above about 10 tokens per second feels fast in a chat. Below 3 tokens per second, the tool becomes frustrating for interactive work, though it is still acceptable for overnight batch jobs.

2.5 The other parts, in brief

Part	Minimum for this course	Recommended	Why
System RAM	16 GB	32 GB	Video editing, browser tabs and the AI tools all compete for RAM. Swap space helps only a little.
Graphics card VRAM	6 GB (learning only)	12 GB or 16 GB	Decides which image, video and LLM models fit.
CPU	4 cores, 2015 or later	6 to 8 modern cores	Needed for editing, subtitles and CPU-only models.
Storage	256 GB SSD with 100 GB free	1 TB NVMe SSD	One image model is 2 to 7 GB, one video model 5 to 30 GB, and video projects are large.
Power supply (desktop)	450 W quality unit	550 to 750 W	Graphics cards under load draw most of their power from it. Cheap units cause crashes.
Cooling	Working fans, clean dust filters	Good airflow and case space	Heat causes slowdowns (throttling) and noise.
Network	Wi-Fi or Ethernet	Gigabit Ethernet	Needed to download models (often 5 to 30 GB each) and to reach the machine from other devices.

WARNING ABOUT LAPTOPS

Laptops often have a weaker graphics chip than the same name on a desktop, and they throttle when hot. Check the laptop's GPU VRAM and its "maximum graphics power" (TGP) in its specification sheet. A laptop with 8 GB of VRAM is workable. A laptop with 4 GB or with only integrated graphics is suitable for learning, not production.

CHAPTER 3

Choosing a budget machine

3.1 Four realistic tiers

Prices change every month, and used markets vary by country. Use the tiers below to decide what kind of machine to look for, then check current local prices. The words "good", "workable" and "not practical" describe what you can expect from the machine when used the way this course describes.

Tier	Typical machine	What it does well	What it cannot do well
0. Learning	Laptop or desktop, 8 GB RAM, no dedicated GPU (integrated graphics only)	Chat with 1 to 3 billion parameter models on the CPU, scripts, planning, basic editing with proxy files, learning all the tools	Images (very slow), any video generation, large models
1. Entry	Desktop with 16 to 32 GB RAM and a used NVIDIA card with 6 to 8 GB VRAM (e.g. RTX 3050, RTX 2060, GTX 1660 Super)	7 to 8 billion parameter models at good speed, voice-overs, SDXL-type images in a few tens of seconds, editing	Large language models above 14B (slow), video generation beyond very short clips
2. Sweet spot	Desktop with 32 GB RAM and a used NVIDIA card with 12 GB VRAM (e.g. RTX 3060 12 GB), or 16 GB VRAM (e.g. RTX 4060 Ti 16 GB)	14 billion parameter models at good speed, modern image models, small video models, Full-HD editing	Largest reasoning models, long high-resolution video
3. Heavy	Used NVIDIA RTX 3090 or 4090 (24 GB VRAM), or a workstation with 32 GB+ RAM	32 billion parameter models, faster video and image work, 4K editing	Still not comparable to cloud GPU farms, and the power bill and noise are high

3.2 Apple Silicon and other options

Apple Silicon Macs (M1 and later) share one pool of memory between the processor and graphics. A Mac with 16 GB or 32 GB of unified memory runs language models well through tools such as Ollama and LM Studio, and image generation works through specific Mac-optimised tools, but usually more slowly than an NVIDIA card with the same memory. Video generation on Macs is generally slower too. AMD and Intel graphics cards work for some tools but support is less complete; NVIDIA is the safest choice for this course.

3.3 Choosing a machine: a decision chart

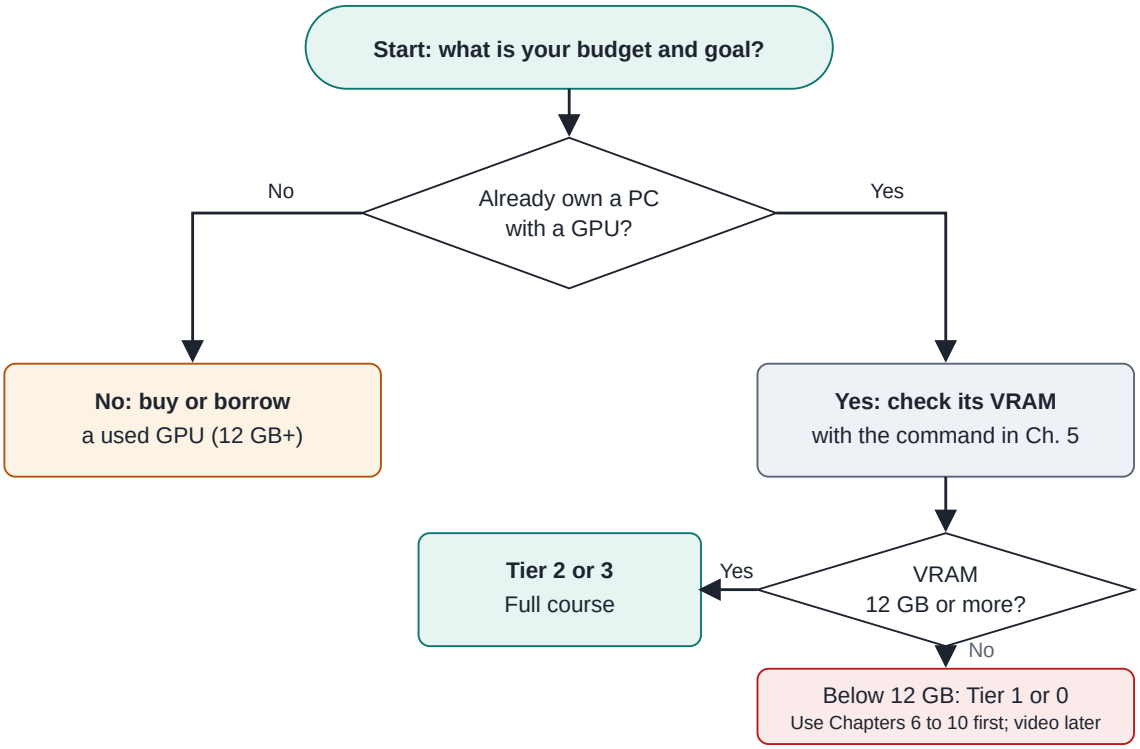


Figure 3.1 A simple chart to pick your starting point. Tier 2 or 3 is the target for full video work.

3.4 What to check before you buy a used machine

- **Graphics card:** confirm the exact model and VRAM from the seller or the product page. Ask for a photo of the label. Watch for mining history (ask directly, and check the fans and heat marks).
- **Power:** a card such as an RTX 3060 needs a 550 W or better power supply with the right connectors. Check the label on the power supply.
- **Case and fit:** measure the card length against the case. Modern cards are often 25 cm or longer.
- **Memory:** buy 16 GB minimum, or two 16 GB sticks for 32 GB. Check the motherboard's maximum supported RAM.
- **Storage:** add a 1 TB NVMe SSD if the machine has a free slot. Models fill disks quickly.
- **Test before paying:** where possible, plug in the card and run a stress test or the commands in Chapter 5.

> **BUDGET IDEA**
>
> Many people already own a gaming PC or a laptop with an NVIDIA card. Start with what you have. Upgrade only after you find the real bottleneck, which Chapter 14 shows you how to measure.

CHAPTER 4

Operating systems: Windows, Linux or macOS

4.1 The short answer

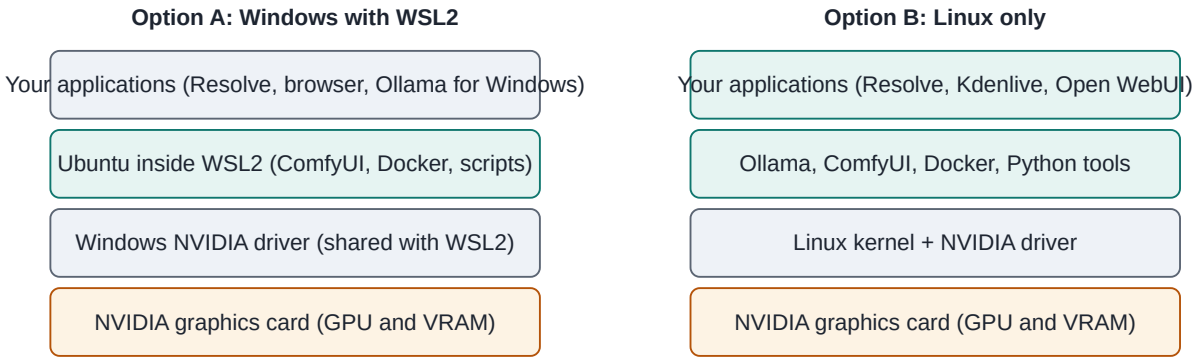
For the tools in this course, the best combinations are:

- **Easiest start, Windows owner:** Windows 11 with the NVIDIA driver, and *Ollama* and *LM Studio* installed directly on Windows. Add *WSL2* (Ubuntu) only when a tool needs Linux.
- **Most reliable for a dedicated AI box:** Ubuntu 24.04 LTS (desktop or server) with the NVIDIA driver from the Ubuntu driver tool.
- **Friendly Linux desktop:** Linux Mint 22 or Pop!_OS (which is especially good with NVIDIA cards).
- **Mac owners:** macOS with Ollama, LM Studio and DaVinci Resolve. Image tools are slower than on NVIDIA.

4.2 Windows versus Linux

Question	Windows 11	Linux (Ubuntu / Mint / Pop!_OS)
Setup effort	Low. Installers for almost every tool.	Medium. Some tools need a terminal and a few commands.
NVIDIA driver	Automatic or from NVIDIA's site.	One command (`ubuntu-drivers`) on Ubuntu; Pop!_OS offers a dedicated NVIDIA ISO.
Image tools (ComfyUI, Forge)	Excellent; many one-click packages.	Excellent; often slightly faster and more stable for heavy jobs.
Video editing	Best choice for DaVinci Resolve and Adobe-style editors.	DaVinci Resolve and Kdenlive work well. Some commercial editors are missing.
Background services and Docker	Possible (Docker Desktop, WSL2), but heavier.	Native, light, and reliable for an always-on server.
Resource use	Higher idle RAM use (often 3 to 4 GB).	Lower idle RAM use (often 1 to 2 GB), which leaves more memory for models.
Updates	Automatic, sometimes at awkward times.	Controlled by you; can break drivers if updated carelessly.
Learning value	Good.	Very good; AI engineering skills transfer straight to cloud servers.

4.3 How the layers fit together



Option A keeps Windows software; Option B gives the least overhead.

In WSL2 never install a separate Linux NVIDIA driver. The Windows driver is shared automatically.

Figure 4.1 Two valid layouts. Pick one, then keep the NVIDIA driver in the correct layer.

4.4 Which Linux distribution?

Distribution	Best for	Notes
Ubuntu Desktop 24.04 LTS	Most people; first Linux system	Five years of standard support. Largest help community. Recommended first choice.
Ubuntu Server 24.04 LTS	Headless homelab box, no screen, controlled remotely	No desktop. Ideal for always-on Ollama and Open WebUI. You need a second computer to manage it, or SSH.
Linux Mint 22	Users who want a classic desktop and a calm, stable base	Based on Ubuntu LTS. Driver setup is through a graphical tool.
Pop!_OS 22.04 or later	NVIDIA owners who want the driver ready out of the box	Offers an NVIDIA-ready image. Good for gaming and creative work.
Debian 12 (stable)	Experienced users who want a very stable server	Slower to adopt new drivers; you may need backports for the newest NVIDIA cards.
Fedora, Arch, openSUSE	Enthusiasts	Newer packages and more manual work. Not recommended as a first AI system.

> **RECOMMENDATION**
>
> If you are unsure: Ubuntu Desktop 24.04 LTS for a single machine you also use for editing, or Ubuntu Server 24.04 LTS for a dedicated headless box. Avoid rolling-release distributions for a system that must keep working while you produce videos.

4.5 Dual boot or single system?

Dual booting (Windows and Linux on one disk, choosing at startup) is possible but has risks: Windows updates can overwrite the boot menu, and the disk layout must be planned carefully. For most learners, either Windows with WSL2 or a separate, dedicated Linux machine is simpler. If you dual boot, back up first and keep at least 100 GB for Linux and 200 GB for models.

CHAPTER 5

Preparing the system, step by step

This chapter prepares the hardware and the operating system. Follow the steps in order. If a step fails, stop and read the troubleshooting note before continuing. Commands are shown for Ubuntu and for Windows with WSL2 where they differ.

5.1 Step 1: Back up and record your starting point

1. Back up your personal files to an external disk or cloud storage.
2. Write down the machine's model, RAM, graphics card, disk size and free space. You will need these numbers in Chapter 14.
3. Plan how many gigabytes you can give to models. A good minimum is 200 GB free on a fast SSD.

5.2 Step 2: Check the BIOS / UEFI settings

Restart and enter the BIOS (usually by pressing Delete, F2 or F10 during startup). Look for and set:

- **Above 4G Decoding:** Enabled (required for many graphics cards with 8 GB or more).
- **Resizable BAR / Smart Access Memory:** Enabled, if available. It can improve speed for some tools.
- **Virtualization (Intel VT-x or AMD-V / SVM):** Enabled, needed for Docker and WSL2.
- **Primary graphics:** set to the dedicated graphics card (PCIe), not the integrated one, if you have a monitor plugged into the NVIDIA card.
- **Power profile:** leave "Balanced" or "Performance" enabled. Do not set a low-power mode that limits the CPU.

5.3 Step 3: Clean and cool the machine

- Open the case, remove dust from the fans, heatsinks and graphics card with compressed air (do it outside, and hold the fans still).
- Make sure the case has at least one intake fan at the front or bottom and one exhaust fan at the back or top.
- For laptops, clean the air vents and place the laptop on a hard, raised surface. Use a cooling stand if the fans are loud.

5.4 Step 4: Install or update the operating system

Windows 11 route: run Windows Update until no updates remain, then install the latest NVIDIA Game Ready or Studio driver from NVIDIA's website, and restart.

Ubuntu route: download Ubuntu Desktop 24.04 LTS (or Server), write it to a USB stick with a tool such as Rufus (Windows), balenaEtcher or the Startup Disk Creator, boot from the USB, and install. Choose "Erase disk" only if the disk is dedicated to Linux. After installation, run:

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

5.5 Step 5: Install the NVIDIA driver (Linux)

Ubuntu can detect and install the best driver automatically:

```
ubuntu-drivers devices           # lists your GPU and the recommended driver
sudo ubuntu-drivers install      # installs the recommended driver
sudo reboot
```

After the reboot, check that the card is active:

```
nvidia-smi
```

The output should show your card name (for example "NVIDIA GeForce RTX 3060"), the driver version, and the memory total (for example "12288MiB"). This is the most important check in the whole chapter. If the command is not found, the driver did not install: run `sudo ubuntu-drivers install` again and reboot, or try Secure Boot settings described in Chapter 14.

5.6 Step 6: Set up WSL2 (Windows users who want Linux tools)

Open PowerShell as Administrator and run:

```
wsl --install -d Ubuntu-24.04
wsl --update
```

Restart when asked, create a Linux user name and password, then open Ubuntu from the Start menu. Inside Ubuntu, check the GPU:

```
nvidia-smi
```

If the card appears, WSL2 is using the Windows driver correctly. Do not install a Linux NVIDIA driver inside WSL2.

5.7 Step 7: Create a sensible folder layout

A tidy layout makes automation (Chapter 13) easy. Create this structure in your home folder. On Windows, use the same names inside your user folder or inside WSL2's Ubuntu home.

```
~/homelab/
  models/            large model files you download yourself
  studio/
    01_ideas/
    02_scripts/
    03_voice/
    04_images/
    05_clips/
    06_projects/     editor project files
    07_exports/      final videos
    08_thumbnails/
    09_logs/         notes about each run and each decision
  docker/            configuration for Open WebUI and other services
```

```
mkdir -p ~/homelab/models ~/homelab/docker
~/homelab/studio/{01_ideas,02_scripts,03_voice,04_images,05_clips,06_projects,07_exports,08
_thumbnails,09_logs}
```

5.8 Step 8: Add swap space (safety net for RAM)

Swap is disk space used when RAM runs out. It is much slower than RAM, so it prevents crashes but does not replace memory. On Ubuntu with 16 GB RAM, a 16 GB swap file is a sensible safety net:

```
sudo fallocate -l 16G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Check it with `free -h`. On Windows, the page file is managed automatically; set it to "System managed" in Advanced System Settings.

5.9 Step 9: Install Docker (optional but recommended)

Docker runs services such as Open WebUI in isolated containers, so they are easy to update and remove. On Ubuntu:

```
sudo apt install -y docker.io docker-compose-v2
sudo usermod -aG docker $USER
newgrp docker
docker run --rm hello-world
```

On Windows, install Docker Desktop and enable the WSL2 backend in its settings.

5.10 Step 10: Install the monitoring tools

You cannot manage what you cannot see. Install tools that show VRAM, RAM and temperature while models run:

```
sudo apt install -y htop btop nvtop        # Ubuntu: CPU, RAM, GPU monitors
watch -n 1 nvidia-smi                       # live GPU view in any terminal
```

On Windows, Task Manager (Performance tab) shows GPU memory and temperature, and `nvidia-smi` works in PowerShell.

5.11 Step 11: Benchmark the machine once

Record a baseline now, so you can compare later. Note the free disk space (`df -h ~`), the free RAM (`free -h`) and the GPU name (`nvidia-smi`). Keep this in `~/homelab/studio/09_logs/baseline.txt`.

SAFETY

Use a surge protector or UPS. A sudden power cut during a model download or a video render can corrupt files. Keep the machine in a ventilated place; a GPU at full load can run 70 to 85 degrees Celsius without throttling, but much hotter means a cooling problem.

CHAPTER 6

Installing the core AI stack and your first chat

6.1 The four building blocks

- **Ollama** (or LM Studio): runs language models and offers a simple local programming interface (API) on your machine.
- **Open WebUI:** a browser-based chat interface that talks to Ollama. It looks like a chat website but runs on your computer.
- **ComfyUI:** a node-based tool for image and video generation (Chapters 9 and 11).
- **Your editor and scripts:** DaVinci Resolve or Kdenlive, plus Python for automation (Chapter 13).

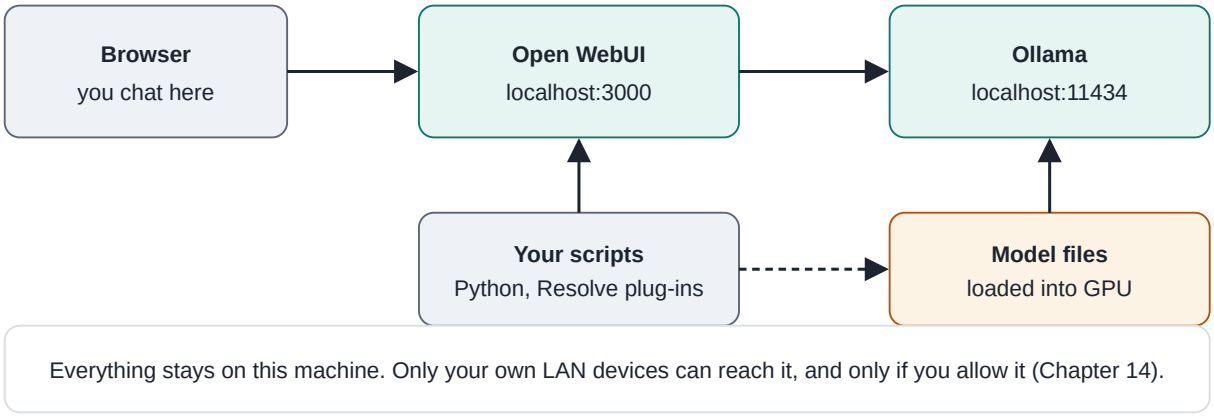


Figure 6.1 How the pieces talk. Scripts can skip the browser and call Ollama directly.

6.2 Install Ollama

Linux (Ubuntu, Mint, Pop!_OS): run the official installer from ollama.com (open the site and copy the current Linux command), then check:

```
ollama --version
systemctl status ollama      # the background service should be "active (running)"
```

Windows: download the Windows installer from ollama.com and run it. It starts in the background and uses the NVIDIA driver automatically.

6.3 Your first model

Start with a small model so the first test is quick. A 3 to 4 billion parameter model works even on an 8 GB laptop:

```
ollama pull llama3.2:3b          # downloads about 2 GB
ollama run llama3.2:3b           # opens a chat in the terminal
```

Type a question, press Enter, and read the answer. Type `/bye` to leave. Then check where the model ran:

```
ollama ps
```

The **PROCESSOR** column matters. "100% GPU" means the model is fully in VRAM, which is what you want. A split such as "48%/52% CPU/GPU" means part of the model is in system RAM and the answer will be slower.

6.4 Add a browser chat with Open WebUI

With Docker installed (Step 9), run:

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui --restart always \
  ghcr.io/open-webui/open-webui:main
```

Open `http://localhost:3000` in your browser, create the local administrator account (stored only on this machine), and choose a model from the menu. The first start takes a few minutes. Open WebUI reaches Ollama through `host.docker.internal:11434`, which the command above sets up.

> **SECURITY**
> Do not expose port 3000 or 11434 to the internet. Ollama has no login by default. On a home network, leave both bound to the local machine unless you have a specific reason, and use a firewall (Ubuntu: `sudo ufw allow from 192.168.1.0/24 to any port 3000` for your own subnet only).

6.5 Try the API from a script

Scripts talk to Ollama over HTTP. This command asks for a short answer and returns JSON:

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2:3b",
  "prompt": "Give three hook sentences for a video about budget home AI.",
  "stream": false
}'
```

In Python, install the official library with `pip install ollama`, then:

```
import ollama

reply = ollama.chat(
    model="llama3.2:3b",
    messages=[{"role": "user", "content": "Write a 20-second intro for a tech channel."}],
)
print(reply["message"]["content"])
```

6.6 Install LM Studio (optional alternative)

LM Studio is a desktop application with a model browser, a chat window and a local server. It is convenient for beginners on Windows and Mac. Install it from lmstudio.ai, search for a model, download it, and load it. You can also start its local server to use the same API style. Do not run LM Studio's server and Ollama on the same port at once; if one is already using port 11434 or 1234, change the other's setting.

6.7 Install ComfyUI (for later chapters)

ComfyUI is installed now so the folders exist when you need them. Windows users can download the portable package from the ComfyUI releases page. Linux users run:

```
cd ~/homelab
git clone https://github.com/comfyanonymous/ComfyUI.git
cd ComfyUI
python3 -m venv .venv
source .venv/bin/activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
pip install -r requirements.txt
python main.py
```

The `cu124` part must match a CUDA version that your driver supports. The PyTorch website generates the exact command for your system; use that if the one above fails. ComfyUI opens at `http://127.0.0.1:8188`. Stop it with Ctrl+C.

6.8 First-day checklist

Check	Command or place	Expected result
GPU visible	`nvidia-smi`	Your card name and memory total
Ollama running	`ollama list`	Lists the model you pulled
Model on GPU	`ollama ps` while chatting	"100% GPU" for a 3B or 7B model
Browser chat works	`http://localhost:3000`	Chat window answers
Script works	Python snippet in 6.5	Prints a short reply
Disk space	`df -h ~`	At least 150 GB free after installs

> **MILESTONE**
> When all six checks pass, the core homelab is working. Everything else in this course builds on this foundation.

CHAPTER 7

Selecting LLM models

7.1 What a model is

A language model is a file of numbers, called *weights*, that has learned patterns from a large amount of text. When you send it a prompt, it predicts the next word (token), again and again, until the answer is complete. Models differ in four ways that matter to you: size, compression, specialisation and licence.

7.2 Size: the parameter count

Parameters are the weights. "7B" means about seven billion parameters. Generally, a larger model follows complex instructions better, makes fewer factual slips and writes more naturally, but it needs more memory and runs more slowly. For YouTube production, the difference between a 7B and a 14B model is noticeable for scripts and planning, and smaller for simple titles or tags.

7.3 Compression: quantization

Models are normally stored with 16 bits per weight. Quantization stores each weight with fewer bits, which shrinks the file and the memory it needs, at a small cost in quality. Ollama and LM Studio usually download a quantized version by default, with names such as `Q4_K_M`. For a 7B to 14B model, Q4_K_M is the usual sweet spot. Q5 and Q6 are slightly better but larger. Q2 and Q3 are often noticeably worse.

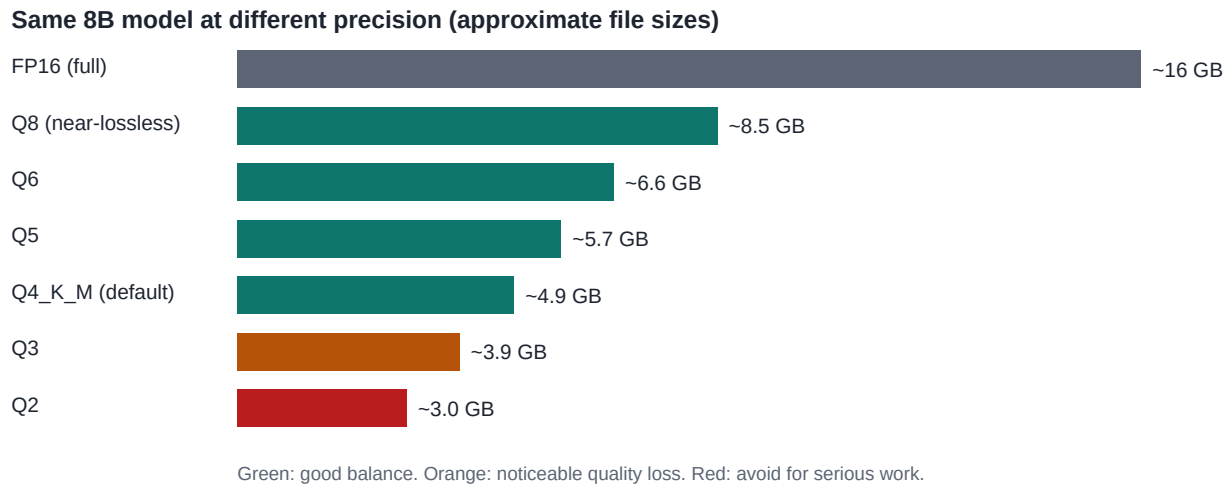


Figure 7.1 Quantization trades file size for quality. Sizes are approximate for an 8B model.

7.4 Specialisation

- **General chat models** (named "instruct" or "chat"): the default for writing, planning and Q&A.
- **Reasoning models:** spend extra tokens "thinking" before answering. Better for hard logic or planning, slower in practice (Chapter 8).
- **Coding models:** trained on source code. Better at writing and fixing programs (Chapter 8).
- **Vision models:** accept images as input, useful for checking thumbnails or describing a frame.

- **Embedding models:** convert text to numbers for search. Not needed for most beginners, but useful if you later build a searchable library of your own scripts.

7.5 Licence: can you use it for a monetised channel?

Model licences differ. Some allow commercial use with simple conditions (for example, Apache 2.0 or MIT). Others are "research only" or "non-commercial", and some ask you to display an attribution. Your channel may earn money, so check the licence of every model you use for final output. Appendix C gives a quick guide. When in doubt, use models under Apache 2.0 or MIT for anything published, and treat others as learning tools only.

7.6 Example model families at the time of writing

Model names and tags change often. Use the names below as starting points and confirm the current tag on the Ollama library page before you download.

Your memory (VRAM or unified)	Chat and writing	Reasoning	Coding
CPU only, 8 GB RAM	llama3.2:3b, gemma3:1b	deepseek-r1:1.5b (basic)	qwen2.5-coder:1.5b
6 to 8 GB VRAM	qwen2.5:7b, llama3.1:8b, gemma3:4b	deepseek-r1:7b or 8b	qwen2.5-coder:7b
12 GB VRAM	gemma3:12b, qwen2.5:14b (about 9 GB, leaves room for context; confirm "100% GPU" in `ollama ps`)	deepseek-r1:14b	qwen2.5-coder:14b
16 GB VRAM	qwen2.5:14b, phi4:14b	deepseek-r1:14b (faster); gpt-oss:20b if it fits	qwen2.5-coder:14b
24 GB VRAM or 32 GB unified Mac	qwen2.5:32b, gemma3:27b	deepseek-r1:32b, gpt-oss:20b comfortably	qwen2.5-coder:32b

Check each model's licence on its page before using it for monetised output. Pull a model with `ollama pull <name>`. Remove an unused one with `ollama rm <name>` to save disk space.

7.7 How to choose: a practical method

1. **Fix your hardware limit.** Use the table above and the memory rule from Chapter 2. Keep 1 to 2 GB free.
2. **Start one size smaller than you think you need** and measure speed with `ollama ps` and by timing a response.
3. **Build a personal test set.** Write five real tasks from your channel: a 60-second hook, a fact list, a title set, a product comparison and a tricky logic question. Save them in `~/homelab/studio/09_logs/tests.txt`.
4. **Run each candidate on the same tests** and score the answers yourself from 1 to 5 for accuracy, style and instruction following.

5. **Choose the smallest model that scores well** on your tasks. A smaller model that is fast and good enough is worth more than a large model you wait for.

> **TIP**
> Public leaderboards are useful for shortlisting, but your own test set decides. A model that tops a general benchmark may still write weak YouTube hooks.

7.8 Context length

The context is how much text the model can consider at once: your prompt, the conversation and the answer. A default of 2,048 to 4,096 tokens is too small for a long script. Increase it in the chat with:

```
ollama run qwen2.5:7b
>>> /set parameter num_ctx 8192
```

Each doubling of the context uses more VRAM. If the model starts splitting between CPU and GPU, lower the context again. One script of 1,500 words is roughly 2,000 tokens, so 8,192 gives room for a script, notes and feedback.

CHAPTER 8

LLMs for deep thinking, coding and scripts

8.1 Three kinds of "thinking" work

Type of work	Best model type	Typical output speed	Use it for
Quick writing	General chat model 7B to 14B	Fast (20 to 60 tokens/s on a good card)	Hooks, outlines, titles, descriptions, rewrites
Deep thinking	Reasoning model (for example deepseek-r1 or gpt-oss)	Slower, because the model writes a long internal reasoning first	Checking logic, planning a multi-part series, finding gaps in an argument, reviewing a script for consistency
Coding	Coding model (qwen2.5-coder)	Fast	Scripts that automate your pipeline, fixing errors, explaining code

8.2 Deep thinking: how to use reasoning models well

Reasoning models show their working in a separate "thinking" section. This makes them better at multi-step problems but also much slower. Use them for a few high-value decisions, not for every sentence.

- **Good uses:** "Here is my script outline. Find three logical gaps a skeptical viewer would notice." Or: "Plan a 6-video series that builds from basics to advanced, with dependencies between videos."
- **Poor uses:** bulk generation of 50 titles, where a fast model is cheaper in time.
- **Tip:** ask for a short final answer. Otherwise the reasoning can run for thousands of tokens.

8.3 A reusable scriptwriter model

Instead of repeating a long instruction, build a named model with a fixed system message. Create a file called `Modelfile` (no extension) in `~/homelab/studio`:

```
FROM qwen2.5:7b
PARAMETER temperature 0.7
PARAMETER num_ctx 8192
SYSTEM """
You write scripts for an educational YouTube channel about practical technology.
Rules:
- Plain English, short sentences, one idea per sentence.
- Open with a hook of two sentences, then state what the viewer will learn.
- Mark each section with a heading in square brackets, for example [Section 2: Setup].
- Never invent statistics, prices or quotes. Write "[CHECK: ...]" where a fact must be
verified.
- Keep the spoken script between 130 and 150 words per minute of video.
"""
```

```
ollama create scriptwriter -f Modelfile
ollama run scriptwriter "Topic: why a used RTX 3060 is a good starting GPU for local AI.
Length: 8 minutes."
```

The "[CHECK: ...]" rule is important. Local models can be confidently wrong, so the script itself points to the claims you must verify before recording.

8.4 Prompts that work for YouTube

Task	Prompt pattern
Outline	"Topic: X. Audience: beginners with a basic laptop. Give 6 sections, each with one takeaway and one example."
Hook options	"Write 5 opening hooks for a video about X. Each under 25 words. Vary the style: question, bold claim, story, surprise, challenge."
Title and thumbnail text	"Give 10 titles under 60 characters and 10 thumbnail phrases of 3 words or fewer. Avoid clickbait claims that the video does not deliver."
Description and chapters	"Write a 150-word description with a chapter list using these timestamps: [paste]. Keep the timestamps exactly."
Script review	"Read this script as a skeptical beginner. List confusing terms, unsupported claims and places where I skip a step."
Shorts from a long video	"From this transcript, pick 3 moments of 30 to 45 seconds that stand alone. Give start and end timestamps and a one-line reason."

8.5 Coding helpers

A coding model can write the small programs that glue your studio together: renaming files, batch generating scripts, checking audio levels, or calling ComfyUI. Three easy ways to use one:

- **Open WebUI:** paste code and ask for a fix or an explanation. Good for beginners.
- **Continue (VS Code extension):** connect it to your Ollama server to get inline suggestions and chat about a file in your editor.
- **Aider (command line):** works on whole project folders. Point it at a local Ollama model with the `ollama/` prefix. Keep changes in version control with Git.

Always read and test generated code before running it on your files. Start with a copy of your folder. Never let an automatic tool delete files without a backup.

8.6 A small batch script that uses the local model

This example reads one idea per line from a text file and writes a draft outline for each into the scripts folder. It uses only the Ollama library installed in Chapter 6.

```
import pathlib
import ollama

ideas = pathlib.Path("~/homelab/studio/01_ideas/ideas.txt").expanduser()
out_dir = pathlib.Path("~/homelab/studio/02_scripts/outlines").expanduser()
out_dir.mkdir(parents=True, exist_ok=True)

for n, line in enumerate(ideas.read_text(encoding="utf-8").splitlines(), start=1):
    if not line.strip():
        continue
    reply = ollama.chat(
        model="qwen2.5:7b",
        messages=[{"role": "user", "content": f"Write a 6-section outline for a video on:
{line}. "
                   "Mark anything that needs fact checking with [CHECK]."}],
    )
    (out_dir / f"outline_{n:03d}.md").write_text(reply["message"]["content"],
encoding="utf-8")
    print("done", n)
```

HUMAN STEP

Every outline and script must be read by you. The model drafts; you decide what is true, fair and suitable for your audience.

CHAPTER 9

Image generation

9.1 What image generation can and cannot do well

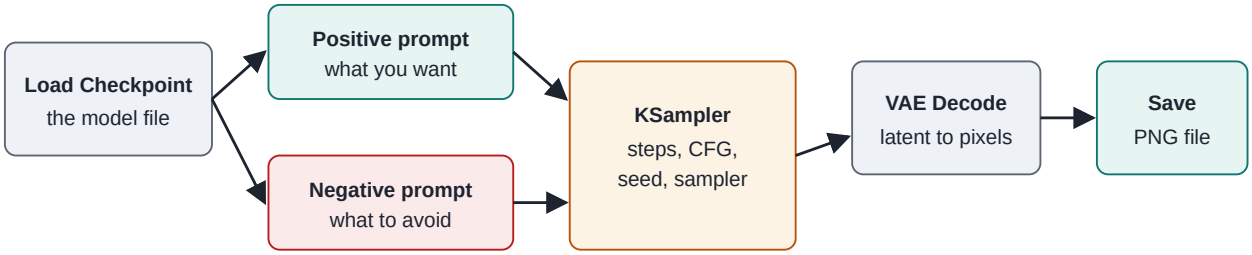
Image models are excellent at backgrounds, scenes, product mock-ups, illustrations and abstract covers. They are weaker at exact text, accurate hands and consistent characters across many images. For thumbnails, generate the background and the subject with AI, then add text, arrows and logos in your editor, where you control the fonts and spelling.

9.2 Model families

Family	Native resolution	VRAM need (approx.)	Licence (check current terms)	Notes
Stable Diffusion 1.5	512 px	4 GB+	CreativeML OpenRAIL-M	Oldest, fastest, huge library of style add-ons. Lower detail.
SDXL and its variants	1024 px	8 GB+	OpenRAIL++ (check each variant)	Good all-round quality on modest hardware.
FLUX.1 schnell	1024 px	12 GB+ (quantized versions lower)	Apache 2.0	Very good detail and prompt following; few steps needed.
FLUX.1 dev	1024 px	12 GB+ (quantized versions lower)	Non-commercial licence	Learning and testing only unless you buy a commercial licence.
Newer efficient models (for example Z-Image-Turbo)	1024 px	8 to 16 GB	Verify on the model page	Fast generation with good quality; check licence before use.

9.3 How the pipeline is built: ComfyUI

ComfyUI shows generation as a graph of boxes called nodes. You do not need to program. You connect the boxes, load a saved graph (called a workflow), and press Queue.



Empty latent image (set width and height, for example 1024 x 1024) feeds the KSampler too.

Every node is a visible step. Click any node to change it; save the whole graph as a workflow file.

Figure 9.1 A basic ComfyUI text-to-image graph. Arrows show data flowing from the model to the saved picture.

9.4 Install and first image

Install ComfyUI as described in Chapter 6.7. Download a checkpoint (the model file) from its official page into `ComfyUI/models/checkpoints/`. Start with an SDXL checkpoint on an 8 GB card, or a FLUX schnell variant on 12 GB. Restart ComfyUI. Load the default workflow, select your checkpoint, enter a prompt, and press Queue.

9.5 Settings that matter

Setting	Starting value	Effect
Width and height	1024 x 1024 (SDXL, FLUX); 512 x 512 (SD 1.5)	Use the native size of the model. Larger sizes need much more VRAM.
Steps	20 to 30 (SDXL); 4 to 8 (schnell or Turbo models, check model card)	More steps are slower, sometimes better, up to a point.
CFG (guidance)	5 to 7 for SDXL; follow the model card for FLUX	Higher means stricter prompt following but can look harsh.
Seed	Fixed number, for example 42	The same seed with the same settings repeats the image. Change it to explore.
Sampler	DPM++ 2M or Euler (default is fine)	Changes the look slightly and the speed.

9.6 Writing prompts that work

A reliable prompt has five parts, in roughly this order:

```
[subject] + [action or setting] + [style] + [lighting] + [camera or composition] + [quality words]

Example:
"A young woman explaining a laptop screen in a bright home office, flat illustration style, soft window light, medium shot, clean shapes, high detail"
```

- **Be concrete:** "a wooden desk with a small GPU fan" beats "technology".

- **Name the audience when people appear.** Models otherwise pick a default look, which may not suit your viewers.
- **Use negative prompts** (in SD and SDXL) for "blurry, extra fingers, watermark, text, signature". FLUX models respond better to positive wording such as "plain background".
- **Keep a prompt log** in `09_logs/`, with the seed, model and settings, so good images can be reproduced.

9.7 Thumbnails

YouTube thumbnails are displayed at 16:9. Use 1280 x 720 pixels as the final size, keep the file under 2 MB, and keep the main subject large and readable at phone size. A practical workflow:

1. Generate a 16:9 background at 1344 x 768 (or a 1024 x 576 image and upscale it).
2. Keep important parts away from the bottom-right corner, where the video length appears.
3. Add the title words, arrows and logo in your editor or an image editor (GIMP, Krita, Photopea in a browser).
4. Export at 1280 x 720 as JPEG or PNG, and check it at phone size.

9.8 Speed and memory tricks

- Start ComfyUI with `python main.py --lowvram` on cards with 6 to 8 GB. Expect slower results.
- Use smaller or quantized (GGUF or fp8) versions of large models if the full model does not fit.
- Generate in batches overnight with a queue. Remove unused models from the checkpoint folder to keep the menu tidy.
- Use the image preview to stop bad images early. Press Cancel instead of waiting.

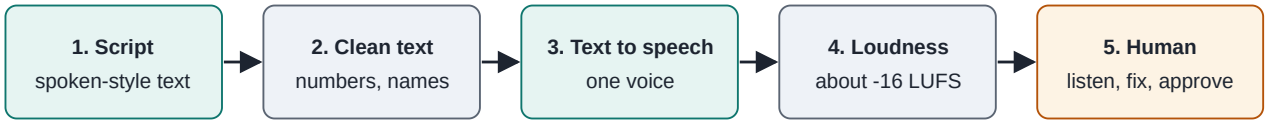
> **RIGHTS AND LIKENESS**
> Do not generate images of real people, public figures or private individuals without clear consent, and never in a way that could mislead viewers. Disclose AI-generated imagery where YouTube's rules or your country's law require it. Keep your generation logs as evidence of how each image was made.

CHAPTER 10

Narration and voice

10.1 The voice chain

A clean voice-over is the product of several small steps. Skipping any one of them is the most common cause of a voice that sounds "robotic" or too loud.



Keep the final WAV (48 kHz, 16-bit) in 03_voice. Export only the approved file to the editor.

Figure 10.1 The narration chain. Step 5 is never skipped.

10.2 Text-to-speech options

Tool	Runs on	Strength	Watch out for
Piper	CPU, very light	Fast, many voices, runs anywhere	Voices sound slightly more basic than newest models. Check each voice's licence.
Kokoro (82M)	CPU or GPU, light	Natural sound for its size; open weights under Apache 2.0	Fewer voice options; verify the model card before commercial use.
F5-TTS	GPU recommended	Voice cloning from a short reference recording	Only clone your own voice or voices you have written consent to use. Check licence.
XTTS-v2 (Coqui)	GPU recommended	Cloning and multiple languages	Licence restricts commercial use; treat as learning-only unless you confirm otherwise.

For a first channel, start with **Kokoro** or **Piper**: both run on modest hardware and give a clean, clear narration. Keep a short test script and compare two or three voices by ear. Most creators choose one voice and keep it for the entire channel, because viewers recognise it.

10.3 A minimal Piper example

Download a Piper voice model (an .onnx file and its .json config) from the Piper voices collection, then:

```
echo "Welcome. Today you will learn how to run a language model on your own computer." \
  | piper --model en_US-lessac-medium.onnx --output_file hook.wav
```

For a long script, split it into paragraphs of about 150 to 300 words, generate one WAV per paragraph, and join them in the editor. Long single runs are harder to fix when one sentence goes wrong.

10.4 Write for the ear

- **Short sentences.** A spoken sentence of 12 to 18 words is easy to follow.
- **Spell out tricky items** the way they should be said: "R-T-X three zero six zero" or "twelve gigabytes", not "RTX 3060 12GB", unless the engine handles it correctly.
- **Use commas and full stops** for pauses. Use an ellipsis sparingly.
- **Keep a pronunciation list** for brand names and technical terms in `03_voice/glossary.txt`, and check it each time.

10.5 Loudness: make every video sound the same

YouTube adjusts playback to about -14 LUFS. Narration at around -16 LUFS integrated, with a true peak at or below -1 dBTP, sounds full on phones and laptops without distortion. Use the free tool FFmpeg for this:

```
ffmpeg -i hook.wav -af loudnorm=I=-16:TP=-1.5:LRA=11 -ar 48000 hook_norm.wav
```

For a more controlled result, use a loudness meter in DaVinci Resolve (Fairlight page) or Audacity with a LUFS plug-in.

10.6 Speech-to-text for subtitles

Speech recognition runs the other way: audio to text. It is used to create subtitles and transcripts for search and for Shorts clipping. The recommended open tool is **Whisper**, especially in the faster-whisper or whisper.cpp versions:

```
pip install faster-whisper

from faster_whisper import WhisperModel

model = WhisperModel("small", device="cuda", compute_type="float16")   # use "cpu" and
"int8" without a GPU
segments, info = model.transcribe("03_voice/final_narration.wav", language="en")
for s in segments:
    print(f"{s.start:7.2f} {s.end:7.2f} {s.text}")
```

Use the `medium` model for better accuracy on technical terms. Always read the subtitle file and correct product names before publishing.

10.7 Listening checklist

Problem	What you hear	Fix
Mispronunciation	A word sounds wrong	Respell it phonetically in the script, regenerate that paragraph only
Clipping or harsh peaks	Crackle on loud words	Lower the gain or re-normalise to -16 LUFS with true peak below -1 dBTP
Robotic rhythm	Every sentence has the same tone	Shorten sentences, add natural pauses, try another voice
Clicks at joins	Small pop between clips	Add a 10 to 20 ms crossfade at each join in the editor

Background hum	Constant noise under speech	Use a clean-up plug-in or re-generate; never use a noisy recording of your own voice without cleaning it

> **TIP**
> Listen to each voice-over on laptop speakers and on earphones. Narration that sounds fine on one can be unclear on the other.

CHAPTER 11

Video generation: what is realistic on a budget

11.1 Honest expectations

AI video generation has improved fast, but it remains slow and demanding. A short clip of about five seconds can take several minutes on a 12 GB card and far longer on weaker hardware. Results may flicker, drift or distort hands and faces. Generated clips are best used as **B-roll** (supporting shots) of a few seconds each, not as the spine of a long video.

11.2 Types of generation

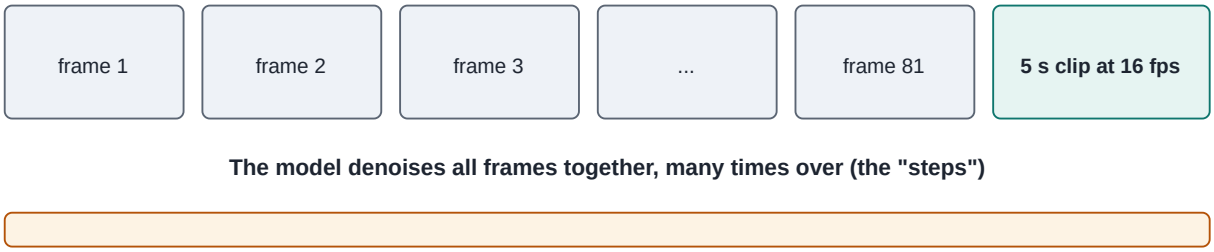
- **Text to video:** write a description, get a clip. Most flexible, least controllable.
- **Image to video:** start from a picture you made or own, and animate it. More control; use this when you need a specific look.
- **Video to video:** restyle an existing clip. Needs strict rights checks because it starts from someone else's footage.

11.3 Model options and rough hardware needs

Model family (examples)	Smallest practical GPU (approx.)	Typical result on budget hardware	Licence note
Wan 2.1 / 2.2 small variants (about 1.3B to 5B)	8 to 12 GB with offloading	Short clips at lower resolution, several minutes each	Check model card before monetised use
LTX-Video	8 to 12 GB	Fast short clips, quality varies with prompt	Check current terms
CogVideoX (2B)	12 GB+ with offloading	Short clips, moderate quality	Check current terms
HunyuanVideo and other large models	24 GB+ or cloud GPU	Best quality, not practical on a 12 GB card	Check current terms

Model names, versions and minimum memory change quickly. Always read the current model card and its example settings before you start a long job.

11.4 What a short clip is made of



Time cost ~ resolution x frames x steps. Halving the resolution or the clip length roughly divides the time.

Use 480p and 3 to 5 seconds for tests; reserve 720p for shots you have approved.

Figure 11.1 A clip is many frames created together. The cost grows with resolution, length and steps.

11.5 A practical workflow

1. **Plan the shot list first.** Each shot should be 3 to 6 seconds, with one clear action.
2. **Make still keyframes** with the image model (Chapter 9) so the look is decided before motion.
3. **Animate at low resolution** (480p) for a test. Discard weak takes quickly.
4. **Re-render the approved take** at the target resolution with the same seed.
5. **Check every clip frame by frame** for hands, faces, text and flicker. Remove any clip with artefacts.
6. **Log the settings** (model, seed, steps, resolution, time taken) in `09_logs/`.

11.6 Better alternatives for most channel needs

- **Licensed stock footage** from reputable libraries, with the licence kept on file.
- **Your own footage:** screen recordings of software, close-ups of your hardware, short phone clips. These are often more trustworthy to viewers.
- **Motion graphics:** animated titles, charts and diagrams made in your editor (Chapter 12), or in Blender for 3D.
- **Still images with motion:** a Ken Burns move (slow pan and zoom) on an AI image costs almost nothing in time.
- **Cloud burst:** for a few final hero shots, rent a cloud GPU for an hour. Compare the cost with your time before you do.

> **DISCLOSURE**
> Realistic AI-generated or altered video that could be mistaken for real events must be disclosed under YouTube's policy. Label stylised or clearly fictional content honestly too. Check the current policy page before each upload.

CHAPTER 12

Video editing and final export

12.1 Choosing an editor

Editor	Price	Platforms	Good for
DaVinci Resolve	Free version is very capable; Studio is paid	Windows, macOS, Linux	Professional editing, colour, audio mixing (Fairlight). Best overall choice.
Kdenlive	Free, open source	Linux, Windows, macOS	Simple, friendly, good proxy support for weaker machines
Shotcut	Free, open source	Windows, macOS, Linux	Lightweight, works on older hardware
OpenShot	Free, open source	Windows, macOS, Linux	Very simple, good for a first video
Blender (video sequencer)	Free, open source	All	Combining 3D or motion graphics with edits

For a Windows or Linux machine with 8 GB or more of VRAM, DaVinci Resolve (free) is the recommended choice. On a weak laptop, Kdenlive or Shotcut with proxies is the better starting point.

12.2 Proxies: editing on a weak machine

A proxy is a small, low-resolution copy of each clip that the editor uses while you cut. The final render still uses the original footage. Proxies let a laptop with 8 GB RAM edit 4K footage smoothly. In Resolve, select the clips in the Media Pool, right-click, and choose Generate Proxy Media (you can also set Proxy Mode in Project Settings). In Kdenlive, enable proxy clips in Project Settings and use the transcode option when importing.

12.3 The timeline

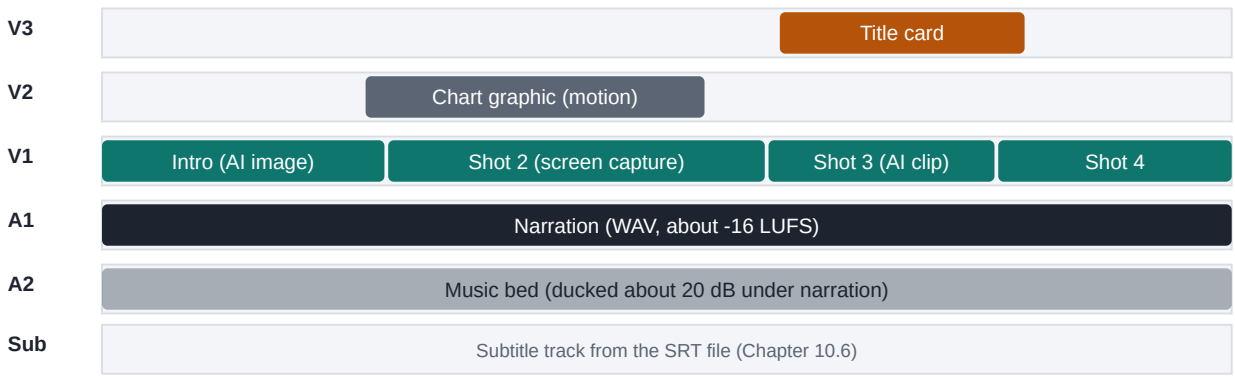


Figure 12.1 A typical timeline. Video on V1, graphics above it, narration and music below, and subtitles at the bottom.

12.4 Editing order that saves time

1. Lay the narration on the timeline first and cut the pauses. The voice sets the pace.
2. Place the shots in order, each shot matched to the sentence it illustrates.
3. Add graphics and titles on the upper tracks.
4. Add the music bed last, at low volume, and duck it under speech.
5. Import the subtitle file and check every line against the audio.
6. Watch the whole video once without stopping. Note every problem, then fix them in one pass.

12.5 Music and sound

Use only music you have the right to use: music you composed, royalty-free music with a licence that covers YouTube monetisation, or music generated with a tool whose terms allow commercial use. Keep the licence and a copy of the track in `08_thumbnails/` or a rights folder. Set the music 18 to 22 dB below the narration, and add a short fade-in and fade-out.

12.6 Export settings for YouTube

Setting	Recommended value
Container	MP4
Video codec	H.264 (widest compatibility); H.265 is optional
Resolution and frame rate	1920 x 1080 at 30 fps (or 24 fps for a cinematic look); 3840 x 2160 for 4K if the source supports it
Video bitrate (SDR)	About 8 Mbps for 1080p30; about 12 Mbps for 1080p60; 35 to 45 Mbps for 4K30
Audio	AAC, stereo, 48 kHz, 192 kbps
Colour	Rec. 709 for standard SDR video

Export from your editor, or use FFmpeg for a clean final encode of the editor's master file:

```
ffmpeg -i final_master.mov -c:v libx264 -preset slow -crf 18 -pix_fmt yuv420p \
  -c:a aac -b:a 192k -ar 48000 -movflags +faststart 07_exports/video_1080p30.mp4
```

On an NVIDIA card, `-c:v h264_nvenc -preset p5 -cq 19` replaces the CPU encoder and is much faster, with slightly larger quality differences.

12.7 Final quality check before upload

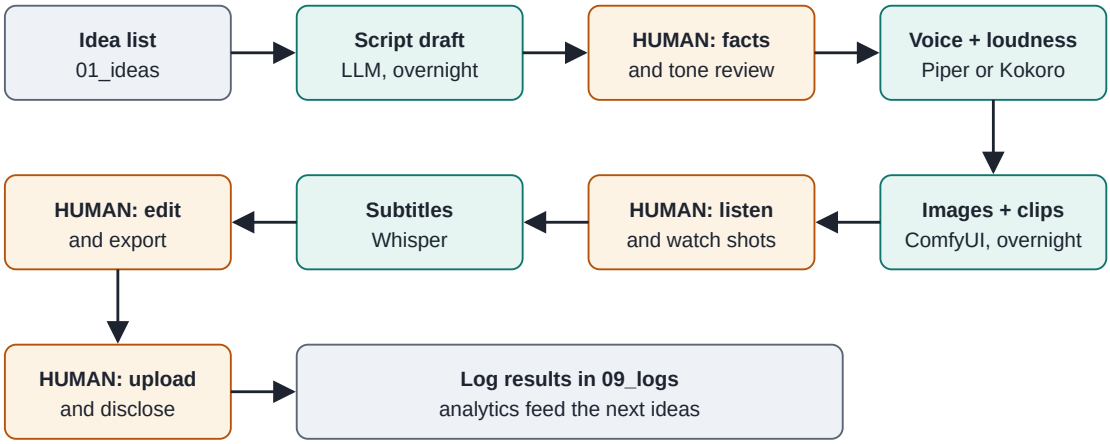
- Play the export on a phone and on a laptop. Check the first five seconds and the last ten.
- Check subtitles, names and numbers against your script.
- Check audio levels: narration clear, music never louder than speech, no clipping.
- Check every AI-generated shot for artefacts, and every stock or music licence on file.
- Confirm the disclosure setting for altered or synthetic content on the upload form.

CHAPTER 13

Automating the YouTube production line

13.1 Think of it as a small factory

Automation does not mean removing people. It means letting the machine do repetitive work overnight and placing a human check at each point where judgment matters: the facts, the tone, the rights, and the final upload. A well-designed line produces more finished videos per month with less stress, while the quality decisions stay with you.



Orange boxes are human decisions. Blue-green boxes run unattended. Nothing reaches YouTube without an orange step.

Figure 13.1 The nightly production line. Machine steps run in the background; human steps are checkpoints.

13.2 Folder naming and a job log

Give every video one project code, for example `V2611-07` (year, month, sequence). Every file for that video starts with the code, so searching is easy and nothing gets mixed up. Keep a simple CSV log in `09_logs/jobs.csv`:

```
code,topic,stage,script_ok,voice_ok,visuals_ok,edit_ok,uploaded,notes
V2611-07,Used GPU buying guide,voice_done,no,no,no,no,no,claims in section 3 need checking
```

A spreadsheet works too. The rule is that each stage has one owner (you, or the machine) and one written status.

13.3 A simple pipeline script

The script below turns a topic into a script, a voice-over and a loudness-normalised narration file. It stops before editing, which remains a human task. It assumes the `scriptwriter` model from Chapter 8 and the Piper voice from Chapter 10. Save it as `~/homelab/studio/pipeline.py` and test it with one topic first.

```
import datetime
import pathlib
import subprocess

import ollama

STUDIO = pathlib.Path.home() / "homelab" / "studio"
VOICE = "en_US-lessac-medium.onnx"          # the Piper voice file you downloaded


def make_script(code: str, topic: str) -> pathlib.Path:
    reply = ollama.chat(
        model="scriptwriter",
        messages=[{"role": "user", "content": f"Topic: {topic}. Length: 8 minutes."}],
    )
    path = STUDIO / "02_scripts" / f"{code}_script.md"
    path.write_text(reply["message"]["content"], encoding="utf-8")
    return path


def make_voice(code: str, script_path: pathlib.Path) -> pathlib.Path:
    # Use only the spoken paragraphs: drop lines that start with "[" (section headings)
    lines = [l for l in script_path.read_text(encoding="utf-8").splitlines()
             if l.strip() and not l.startswith("[")]
    text = "\n".join(lines)
    raw = STUDIO / "03_voice" / f"{code}_raw.wav"
    norm = STUDIO / "03_voice" / f"{code}_narration.wav"
    subprocess.run(
        ["piper", "--model", VOICE, "--output_file", str(raw)],
        input=text.encode("utf-8"), check=True,
    )
    subprocess.run(
        ["ffmpeg", "-y", "-i", str(raw), "-af", "loudnorm=I=-16:TP=-1.5:LRA=11",
         "-ar", "48000", str(norm)],
        check=True,
    )
    return norm


def log(message: str) -> None:
    stamp = datetime.datetime.now().isoformat(timespec="seconds")
    with open(STUDIO / "09_logs" / "pipeline.log", "a", encoding="utf-8") as f:
        f.write(f"{stamp} {message}\n")


if __name__ == "__main__":
    code, topic = "V2611-07", "Used GPU buying guide for local AI"
    script = make_script(code, topic)
    log(f"{code} script written: {script.name}")
    narration = make_voice(code, script)
    log(f"{code} narration ready: {narration.name} (human listen required)")
```

Schedule it for the night with `crontab -e`, adding a line such as `0 1 * * * /usr/bin/python3 /home/you/homelab/studio/pipeline.py`. Run it manually first. Check the log and the files each morning.

> **READ THE CODE BEFORE YOU RUN IT**
>
> The script above is a teaching example. Test it on a copy of your folder, and check the paths for your own username and disk. Never let a script delete or overwrite finished work without a backup.

13.4 Optional: visual workflow tools

If you prefer clicking to coding, a self-hosted workflow tool such as n8n can run the same steps with a visual editor. It can be started with Docker (check the current command on the n8n documentation site), and it can call Ollama and ComfyUI over their local addresses. Keep such tools on your private network and protect them with a strong password.

13.5 Quality gates

Gate	Who	Pass condition
Script	Human	Every number, name and claim is verified; every [CHECK] tag is resolved
Voice	Human	Listened to end to end; pronunciation list applied; loudness recorded
Visuals	Human	No artefacts in AI shots; licences on file; no unauthorised likeness
Edit	Human	Watched once without stopping; subtitles checked
Upload	Human	Title and thumbnail truthful; disclosure set; end screen and cards checked

13.6 Policy and ethics for a monetised channel

- **Original value.** YouTube's monetisation rules restrict mass-produced or repetitious content. A channel that only republishes machine text, with little added by you, is at risk. Add your own research, examples, experience and opinion to every video.
- **Do not copy others.** Do not download, re-upload or closely re-narrate other creators' videos. Downloading tools may breach the platform's terms; use only content you have rights to.
- **Disclose synthetic media.** Realistic altered or synthetic content must be labelled in the upload settings where required. Keep a record of how each video was made.
- **Respect voices and faces.** Clone or depict people only with written consent.
- **Accuracy first.** Health, finance and legal topics need extra checking and a clear statement that the video is general information.
- **Copyright on outputs.** Keep licences for every model, font, music and stock item in a rights folder, and check the attribution requirements of each.

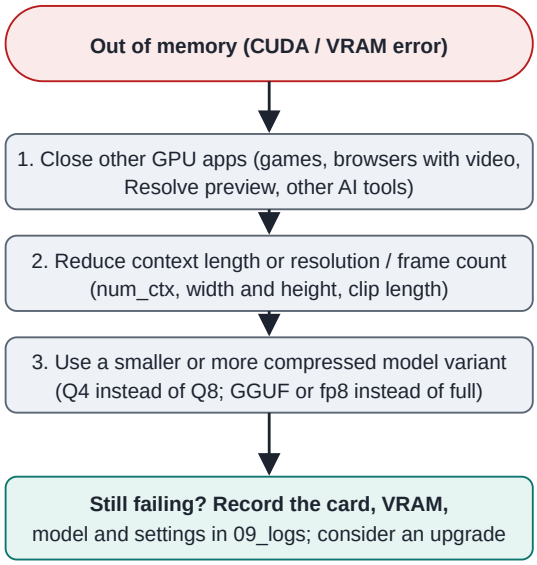
CHAPTER 14

Hardware limits, troubleshooting and performance

14.1 The hard limits, in one table

Limit	What happens	Practical answer
VRAM is too small for the model	Out-of-memory error, or very slow answers from partial CPU use	Choose a smaller model or a more compressed version; shorten context; upgrade VRAM
System RAM too small	Freezes, swapping to disk, programs closing	Upgrade to 32 GB; close browsers and editors during generation; add swap
Memory bandwidth	Speed capped by the memory path, not the chip	Use GPU memory; avoid CPU-only for big models
Heat and power	Clock speeds drop (throttling); fans noisy	Clean dust, improve airflow, limit power, use a cooling pad for laptops
Disk space and speed	Full disk stops downloads; slow disk slows model loading	Keep 100 GB free; use an NVMe SSD for models and projects
Video generation time	Minutes per short clip	Plan shots, test at low resolution, batch overnight, mix with other footage
Model quality ceiling	Small models make more mistakes and sound more generic	Give better instructions, check output, use a larger model for the hardest tasks
Electricity cost	A 300 W load for 2 hours uses 0.6 kWh	Schedule heavy jobs; use lower power limits; switch off when idle

14.2 When a generation fails: a decision flow



Each step is a deliberate change. Change one thing at a time so you know what fixed it.

Figure 14.1 Fix memory errors one step at a time, and keep a record of each change.

14.3 Troubleshooting table

Symptom	Likely cause	Fix
`nvidia-smi` not found (Linux)	Driver not installed, or Secure Boot blocks the module	Run `sudo ubuntu-drivers install` and reboot. If Secure Boot is on, enroll the signing key when the blue MOK screen appears at boot, or disable Secure Boot in the BIOS
`nvidia-smi` not found (WSL2)	Old Windows driver or old WSL	Update the NVIDIA driver on Windows; run `wsl --update`, then `wsl --shutdown` and reopen Ubuntu
Ollama uses CPU only	No GPU detected, or VRAM too small	Check `nvidia-smi`; check `ollama ps`; choose a smaller model
Very slow answers after a while	Thermal throttling or power limit	Check temperatures and clocks in `nvidia-smi`; clean fans; improve airflow
Open WebUI cannot see models	Docker cannot reach Ollama	Use the `--add-host` option from Chapter 6; check that Ollama listens on the right address
Port already in use	Another program (often LM Studio) uses the port	Stop one of the two, or change its port
Disk full during download	Models accumulate	List with `ollama list`, remove with `ollama rm`; move models with `OLLAMA_MODELS`
Machine freezes with large jobs	RAM exhausted	Close apps; add swap; reduce batch size; upgrade RAM

Video output flickers or morphs	Model limit or too many steps for the content	Shorter shots, simpler motion, keyframe-based image-to-video, licensed footage for complex shots
Voice clips cut off at joins	Piper or TTS split a sentence	Split the text at full stops; add small crossfades in the editor

14.4 Performance settings worth knowing

Ollama reads a few environment variables. On Ubuntu, add them to the service with `sudo systemctl edit ollama`, then write the lines under a `[Service]` heading, save, and run `sudo systemctl restart ollama`:

```
[Service]
Environment="OLLAMA_KEEP_ALIVE=10m"          # keep a model loaded 10 minutes after use
Environment="OLLAMA_MAX_LOADED_MODELS=1"     # one model in memory at a time: fewer
surprises
Environment="OLLAMA_NUM_PARALLEL=1"          # one request at a time: fastest single
answers
Environment="OLLAMA_FLASH_ATTENTION=1"       # faster attention on supported cards
Environment="OLLAMA_MODELS=/mnt/models/ollama"   # optional: store models on another disk
```

KEEP IT PRIVATE

Ollama listens only on this machine by default. Setting `OLLAMA_HOST=0.0.0.0` makes it reachable from your whole network. Only do this behind a firewall, and never forward it through your router to the internet.

14.5 Lowering power and heat (NVIDIA)

You can cap a card's power to reduce heat and noise, for a small loss in speed. Check the allowed range first:

```
nvidia-smi -q -d POWER | grep -i "power limit"
sudo nvidia-smi -pl 150          # sets a 150 W limit (example for an RTX 3060-class card)
```

This setting resets at reboot. Test a few jobs before and after to measure the trade-off in time and temperature. Record both in your log.

14.6 Measuring what you have

Measure	Tool	Good result
Tokens per second	`ollama run model --verbose` (shows eval rate)	7B model: over 30 tok/s on a 12 GB card; 14B: over 15 tok/s
GPU memory in use	`nvidia-smi`	Below total VRAM with 1 to 2 GB spare
GPU temperature	`nvidia-smi`	Under about 83 degrees Celsius under load

Model placement	`ollama ps`	"100% GPU" for the model you use most
Video render time	Your own log	Trend should not get worse over months (dust build-up shows here first)

Figures are rough guides, not guarantees. Your own log is the only result that matters for your work.

14.7 When to upgrade, and what to buy first

1. **RAM first** if you see freezes or swapping, and it is cheap.
2. **Disk second** if you are short of space: a 1 TB NVMe drive solves most problems.
3. **VRAM third**, when you consistently need a model or image size that does not fit. This is the most expensive upgrade and the one with the biggest effect.
4. **A cloud GPU** for occasional heavy jobs, if the upgrade is not justified by your monthly volume.

CHAPTER 15

Career path, budget plan and 12-week study plan

15.1 Realistic career paths

A local AI studio is a skill base, not an automatic income. The people who earn from it combine tools with editing judgment, storytelling and reliable delivery. Common paths include:

- **Faceless or semi-faceless channel producer:** you build and run your own channel with a clear niche.
- **Freelance video editor** for creators and small businesses, using AI tools for drafts, subtitles and voice-overs.
- **Explainer video producer** for companies, schools and non-profits.
- **Thumbnail, graphic and audio cleaning services** for other creators.
- **Creator automation specialist:** setting up safe production lines for small teams.

Platform income is uncertain. YouTube's Partner Program requires minimum subscriber and watch-time thresholds, and these change, so check the current requirements before you plan around them. Most channels take many months to grow. Freelance and service income is often more predictable, and it can fund the channel.

15.2 Skills to build alongside the tools

Skill	Why it pays	How to practise
Storytelling and scripting	Viewers stay for the story, not the tool	Write one script a week and review it against your best-performing video
Editing rhythm and sound	Good audio makes average visuals watchable	Re-edit one old video a month with a stricter pace
Thumbnail and title design	Click-through rate depends heavily on these	Make three thumbnail variants for each video and compare results honestly
Analytics reading	Tells you what to make next	Review retention graphs weekly and write one lesson per video
Basic Python and command line	Automates repetitive production	Automate one small chore a month
Rights and disclosure	Protects income and reputation	Keep a rights folder and a one-page policy for your channel

15.3 Budget plan: four levels

Costs depend on your country and the used market. The bands below describe the relative spend. Check current prices before buying.

Level	What you buy	Relative cost	What it unlocks
A. Start	Nothing new. Ubuntu or Windows on your existing machine; free tools (Ollama, Open WebUI, Kdenlive or Resolve, FFmpeg, Piper)	Free, plus electricity	Chapters 5 to 10 and basic editing. Small models, voice-overs, scripts
B. Add RAM and storage	RAM to 32 GB; 1 TB NVMe SSD; surge protector	Low	Bigger models, smoother editing, more room for projects
C. Add a used 12 GB GPU	Used NVIDIA card with 12 GB VRAM; a quality power supply if needed	Medium	14B models, modern image models, short video tests, Full-HD production
D. Add a 24 GB GPU	Used 24 GB card and a suitable case and power supply	High	Larger models, faster image and video, 4K editing

Rule of thumb: do not buy hardware for a skill you have not yet practised. Complete Level A for at least one month, record which task is slowest, and then buy only the part that removes that bottleneck.

15.4 Running costs

```
electricity per hour (kWh) = power draw (W) ÷ 1000
cost per session           = kWh × your price per kWh

Example: 300 W for 2 hours = 0.6 kWh.
At an electricity price of 0.20 per kWh, that is 0.12 per session.
Idle at 60 W for 24 hours = 1.44 kWh per day.
```

Heavy jobs are cheap per hour, but they add up over a month. Run long batches at night when you are not using the machine, and turn the machine off or let it sleep when idle.

15.5 A 12-week study plan

This plan assumes about 6 to 8 hours per week. Each week ends with a deliverable you can check.

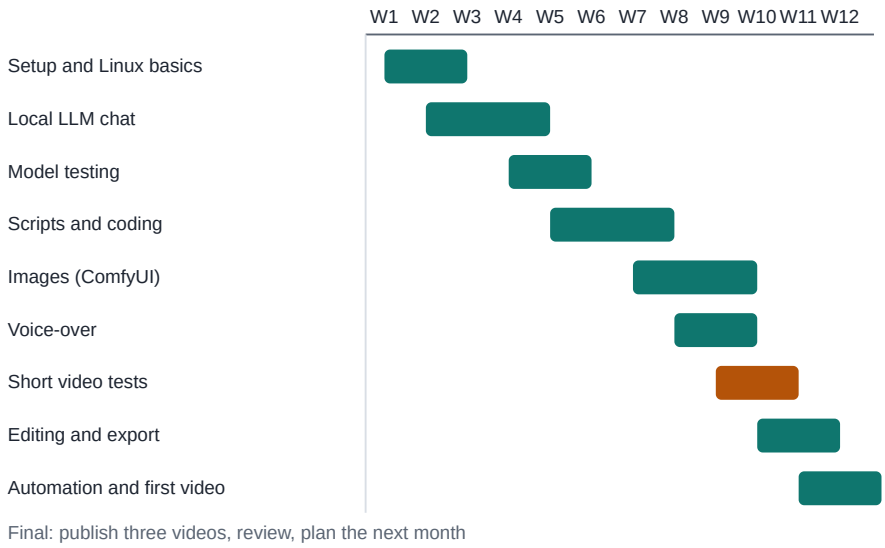


Figure 15.1 Study timeline. Each bar is a skill block; the orange bar marks the most demanding, where most learners need extra time.

Week	Focus	Deliverable (check it before moving on)
1	Operating system, drivers, folders, monitoring tools	`nvidia-smi` works; folder layout exists; baseline logged
2	Ollama and Open WebUI; first chats; `ollama ps`	Five answers on your topic; model placed on GPU
3	Model comparison with your test set (Chapter 7)	Scored table of three models; one chosen with reasons
4	Scripts: scriptwriter model, prompt patterns, batch script	Three outlines and one full script with [CHECK] tags resolved
5	ComfyUI, first images, prompt log	Ten images, with logs; one thumbnail background
6	Image settings, thumbnails at 1280 x 720	Three finished thumbnail variants compared
7	Voice: Piper or Kokoro, loudness, pronunciation list	One approved narration at about -16 LUFS
8	Subtitles with Whisper; voice QA checklist	Accurate SRT file for your narration
9	Short AI video tests at 480p	Three test clips logged with time taken and artefacts noted
10	Editing: proxies, timeline, music ducking	A 2-minute edited sequence
11	Automation: pipeline script, job log, quality gates	One topic run end to end with human checkpoints
12	Full video: script to export, check, upload with disclosure	A finished, published video and a written review of what to change

FINAL ADVICE

The most valuable output of this course is not the videos. It is your log: a record of what you tried, what it cost in time and power, and what viewers responded to. Keep it honest, and your next purchase will be the right one.

APPENDIX A

Command cheat sheet

Purpose	Command
Show GPU, VRAM and temperature	`nvidia-smi` (or `watch -n 1 nvidia-smi`)
Install recommended NVIDIA driver (Ubuntu)	`sudo ubuntu-drivers install`
Check RAM and swap	`free -h`
Check disk space	`df -h ~`
Find the largest folders	`du -sh ~/* | sort -h`
Download a model	`ollama pull qwen2.5:7b`
Chat in the terminal	`ollama run qwen2.5:7b`
List installed models	`ollama list`
Show loaded models and placement	`ollama ps`
Remove a model	`ollama rm model-name`
Create a custom model from a Modelfile	`ollama create scriptwriter -f Modelfile`
Set context in a chat	`/set parameter num_ctx 8192`
Restart Ollama (Linux)	`sudo systemctl restart ollama`
Start Open WebUI	`docker start open-webui`
Start ComfyUI	`cd ~/homelab/ComfyUI && source .venv/bin/activate && python main.py`
Generate speech	`echo "text" | piper --model voice.onnx --output_file out.wav`
Normalise loudness	`ffmpeg -i in.wav -af loudnorm=I=-16:TP=-1.5:LRA=11 -ar 48000 out.wav`
Final MP4 encode (CPU)	`ffmpeg -i master.mov -c:v libx264 -preset slow -crf 18 -c:a aac -b:a 192k -movflags +faststart out.mp4`
Final MP4 encode (NVIDIA)	`ffmpeg -i master.mov -c:v h264_nvenc -preset p5 -cq 19 -c:a aac -b:a 192k out.mp4`
Set GPU power limit (NVIDIA, resets at reboot)	`sudo nvidia-smi -pl 150`

APPENDIX B

Glossary

Term	Meaning
Checkpoint	A saved model file for image generation (weights).
Context	The amount of text a language model can consider at once, measured in tokens.
CUDA	NVIDIA's software layer that lets programs use the GPU. Installed with the driver for most tools.
Diffusion	The method most image and video models use: start from noise and refine it step by step.
Docker	A tool that runs programs in isolated packages called containers.
Fine-tuning and LoRA	Training a small add-on on your own examples to adjust a model's style. Needs more hardware and care.
FFmpeg	A free command-line tool for converting, scaling, joining and encoding media.
GPU, VRAM	The graphics processor, and its dedicated memory. VRAM is the most important limit for AI work.
Inference	Running a trained model to get an answer or output. This is what your homelab does.
LUFS	Loudness Units relative to Full Scale: the standard measure of perceived loudness.
LLM	Large language model: a model that generates and understands text.
Parameters	The learned numbers in a model. "7B" means seven billion.
Proxy	A low-resolution copy of footage used for smooth editing on weak machines.
Quantization	Storing model weights with fewer bits to reduce size and memory, at some quality cost.
Reasoning model	A model that writes internal step-by-step thinking before its final answer.
Seed	A number that controls randomness. The same seed and settings give the same result.
Swap	Disk space used as slow overflow memory when RAM runs out.
Throttling	Automatic slowdown of a processor to avoid overheating or exceeding its power limit.
Token	A chunk of text, often a word or part of a word, that a model reads or writes.
TTS / STT	Text-to-speech (voice generation) and speech-to-text (transcription).
WSL2	Windows Subsystem for Linux, version 2: a real Linux kernel inside Windows.

APPENDIX C

Licence quick reference

This guide is a starting point, not legal advice. Always read the full licence and model card before publishing.

Licence type	Commercial use	Typical conditions	Examples (verify)
Apache 2.0	Yes	Keep the notice; state changes if you modify	Many newer models and tools
MIT	Yes	Keep the copyright notice	Many libraries and some voice tools
OpenRAIL family	Usually yes	Use restrictions on harmful or misleading use apply	Stable Diffusion family variants
Community or custom licence	Often yes, with limits	Revenue thresholds, attribution, acceptable-use rules	Some image and audio models
Non-commercial (CC BY-NC or similar)	No	Learning, testing and personal use only	Some high-quality image and voice models
Research-only	No	Research use only	Some experimental video and voice cloning models

BEFORE EVERY RELEASE

Keep a file called `rights.txt` in each project folder. For every model, voice, font, image and music track used in a published video, record its name, version, licence, and the date you checked it.